

Final Project – DevOps-Adrian-Dmytryk

Spis treści

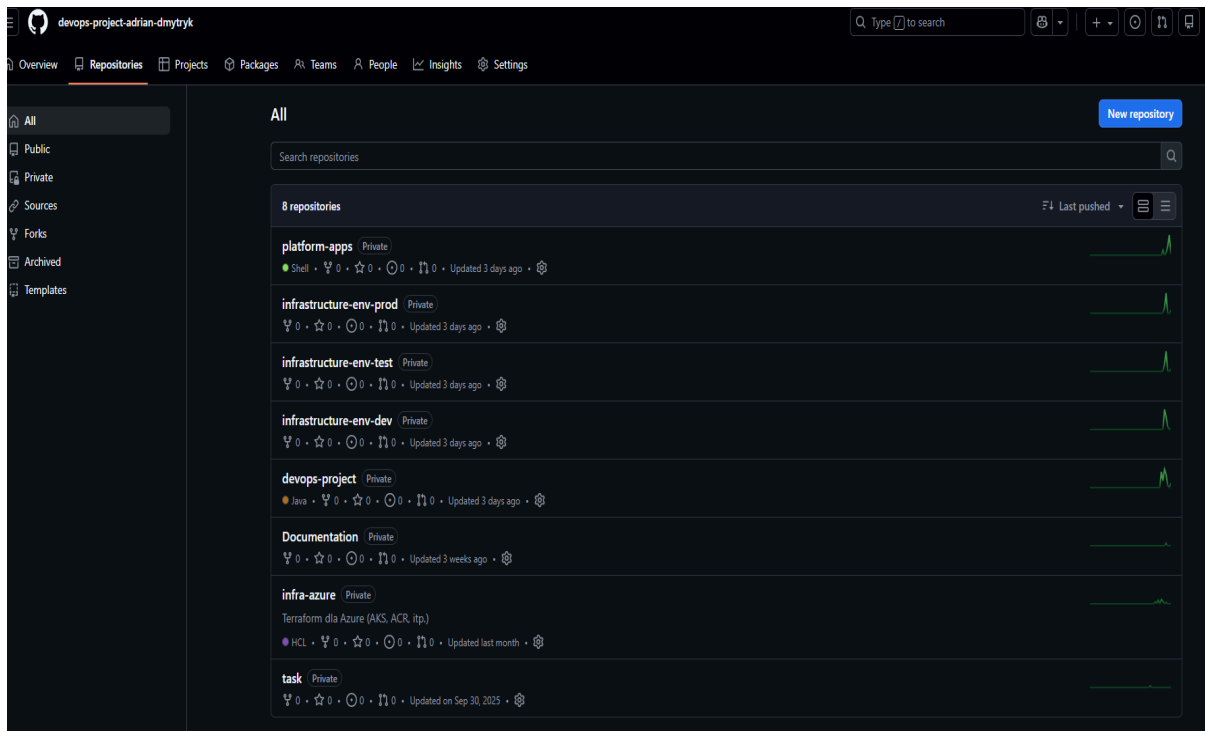
- [1. Przygotowanie organizacji](#)
 - [1.1. Utworzenie repozytorium](#)
 - [1.2. Konfiguracja GitHub Apps](#)
- [2. Infra-azure – Infrastruktura jako kod](#)
 - [2.1. Struktura projektu](#)
 - [2.2. Opis plików głównych](#)
 - [2.3. Moduły Terraform](#)
 - [2.4. Skrypty automatyzacji](#)
 - [2.5. Proces wdrożenia](#)
 - [2.6. Instalacja ArgoCD](#)
- [3. Platform-apps – GitOps i App-of-Apps](#)
 - [3.1. Struktura projektu](#)
 - [3.2. Folder bootstrap](#)
 - [3.3. Folder charts/app-of-apps](#)
 - [3.4. Skrypty automatyzacji](#)
 - [3.5. Rezultat wdrożenia](#)
- [4. Przygotowanie narzędzi CI/CD](#)
 - [4.1. Konfiguracja SonarQube](#)
 - [4.2. Konfiguracja DependencyTrack](#)
 - [4.3. Konfiguracja kube-prometheus-stack](#)
- [5. Devops-project – Aplikacja i CI Pipeline](#)
 - [5.1. Zakres CI Pipeline](#)
 - [5.2. Etapy wykonania](#)
 - [5.3. Przepływ CI/CD](#)
 - [5.4. Konfiguracja sekretów GitHub](#)
 - [5.5. Uruchomienie CI](#)
 - [5.6. Wyniki analizy](#)
- [6. Infrastructure-env-dev – Środowisko deweloperskie](#)
 - [6.1. Zawartość repozytorium](#)
 - [6.2. Proces GitOps](#)
 - [6.3. Promocja do TEST](#)
- [7. Infrastructure-env-test – Środowisko testowe](#)
 - [7.1. Zawartość repozytorium](#)
 - [7.2. Proces GitOps](#)
 - [7.3. Promocja do PROD](#)
- [8. Infrastructure-env-prod – Środowisko produkcyjne](#)
 - [8.1. Zawartość repozytorium](#)
 - [8.2. Proces GitOps](#)
 - [8.3. Walidacja manifestów](#)
- [9. Monitoring aplikacji](#)

- [9.1. Kluczowe komponenty](#)
 - [9.2. Instrukcja konfiguracji](#)
 - [9.3. Checklist weryfikacji](#)
 - [9.4. Rezultaty monitoringu](#)
 - [10. Dokumentacja projektu](#)
-

Przygotowanie organizacji

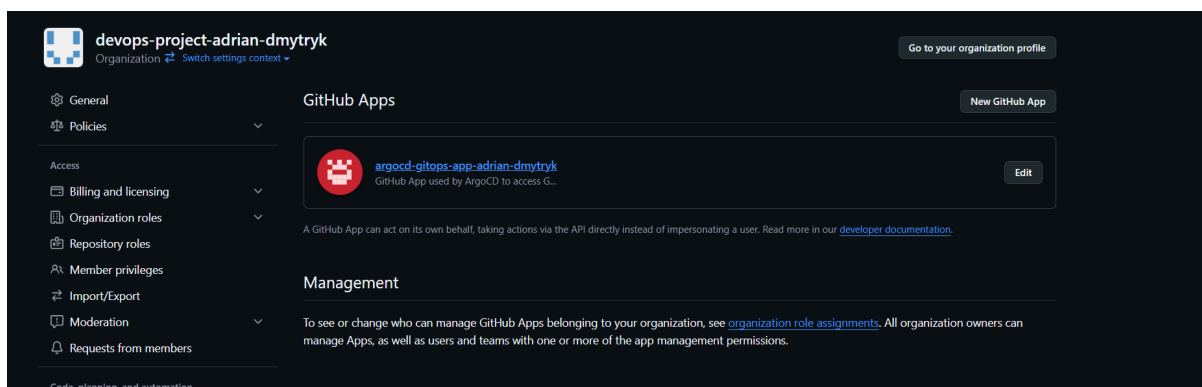
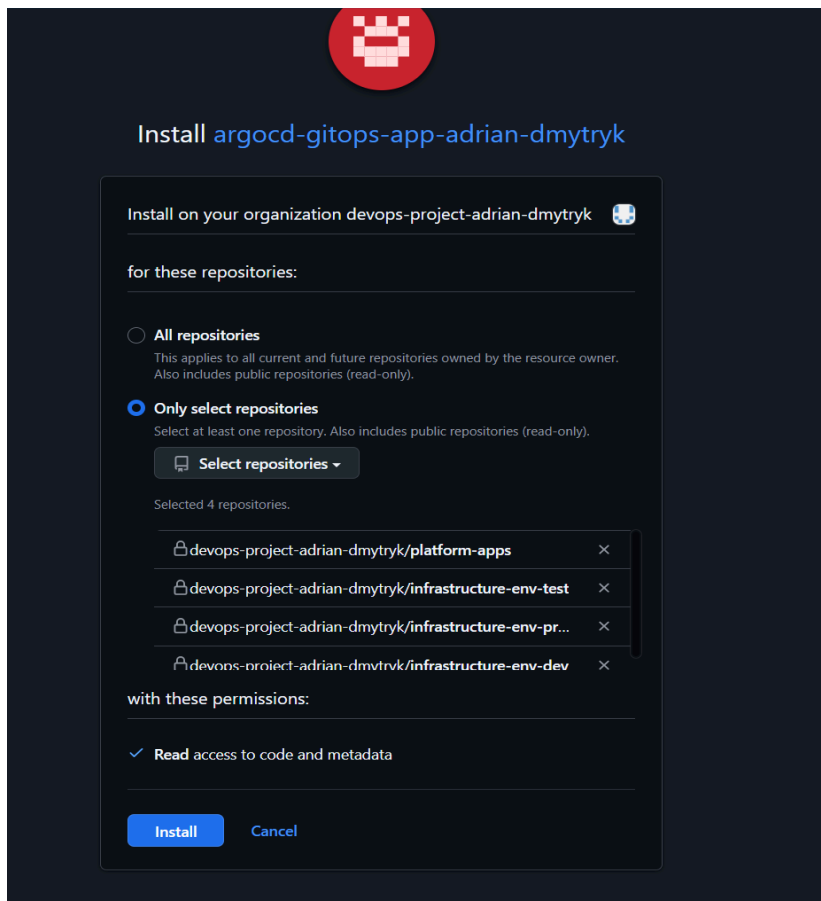
Utworzenie repozytorium

Pierwszym krokiem było utworzenie repozytorium w organizacji GitHub, które będzie stanowiło fundament dla całego projektu DevOps.



Konfiguracja GitHub Apps

W celu autoryzacji repozytorium w ArgoCD, utworzono GitHub Apps dla organizacji. Konfiguracja aplikacji umożliwia bezpieczne połączenie między ArgoCD a repozytoriami organizacji.



Infra-azure – Infrastruktura jako kod

Repozytorium **infra-azure** odpowiada za utworzenie całej infrastruktury wymaganej do realizacji projektu przy użyciu Terraform.

Struktura projektu

└─ backend.hcl

```

├── check_infra.sh
├── deploy.sh
├── destroy.sh
├── envs
│   ├── prod.tfvars
│   └── test.tfvars
├── install_argocd.sh
├── main.tf
├── modules
│   ├── acr
│   │   ├── main.tf
│   │   └── variables.tf
│   ├── aks
│   │   ├── main.tf
│   │   └── variables.tf
│   ├── argocd
│   │   ├── main.tf
│   │   └── terraform.tf
│   ├── auto-shutdown
│   │   ├── main.tf
│   │   └── variables.tf
│   ├── network
│   │   ├── main.tf
│   │   └── variables.tf
│   └── resource-group
│       ├── main.tf
│       └── variables.tf
├── outputs.tf
├── providers.tf
├── README.md
├── terraform.tfstate
├── terraform.tfstate.backup
├── variables.tf
└── versions.tf

```

2.2. Opis plików głównych

Plik	Opis
<code>main.tf</code>	Wywołuje utworzone moduły z folderu <code>./modules</code>
<code>variables</code> <code>.tf</code>	Definicje zmiennych wymaganych dla całego projektu (np. nazwa projektu, region)
<code>outputs.t</code> <code>f</code>	Określa informacje wyświetlane po zakończeniu <code>terraform apply</code> (np. adres IP klastra)

`providers` Konfiguracja dostawców (np. `azurerm` dla Azure, `kubernetes`)
`.tf`

`versions.`
`tf` Określa wymagane wersje Terraform i dostawców dla zapewnienia kompatybilności

2.3. Moduły Terraform

Folder `./modules` zawiera niezależne jednostki infrastruktury:

- **resource-group** – Tworzy logiczny kontener na wszystkie zasoby w Azure
- **network** – Konfiguracja sieci VNET, podsieci (subnets) i reguł bezpieczeństwa (NSG)
- **acr (Azure Container Registry)** – Prywatny rejestr dla obrazów Docker
- **aks (Azure Kubernetes Service)** – Konfiguracja klastra Kubernetes (node pools, wersja klastra)
- **auto-shutdown** – Polityka oszczędzania kosztów (np. wyłączanie maszyn w nocy)
- **argocd** – Moduł instalujący narzędzie GitOps wewnątrz klastra

2.4. Skrypty automatyzacji

Skrypt	Funkcja
<code>deploy.sh</code> / <code>destroy.sh</code>	Automatyzują proces tworzenia i usuwania całej infrastruktury
<code>check_infra.sh</code>	Weryfikuje poprawność utworzonych zasobów
<code>install_argocd.sh</code>	Konfiguruje ArgoCD po utworzeniu klastra (komendy <code>kubectl</code> lub <code>helm</code>)

2.5. Proces wdrożenia

Przebieg wdrożenia infrastruktury:

1. Uruchomienie skryptu `deploy.sh`
2. Skrypt wywołuje `main.tf`, ładując dane z `envs/test.tfvars`
3. `main.tf` uruchamia każdy moduł, przekazując zmienne
4. Terraform zapisuje stan w `terraform.tfstate`

Outputy po wdrożeniu:

Apply complete! Resources: 0 added, 4 changed, 0 destroyed.

Outputs:

```
acr_login_server = "acrfordevops poc01adrian.azurecr.io"
aks_prod_kube_config = <sensitive>
aks_prod_name = "devops-poc01-prod"
aks_test_kube_config = <sensitive>
aks_test_name = "devops-poc01-test"
rg_name = "rg-devops-poc01"
```

Apply complete! Resources: 16 added, 0 changed, 0 destroyed.

Outputs:

```
acr_login_server = "acrfordevops poc01adrian.azurecr.io"
aks_prod_kube_config = <sensitive>
aks_prod_name = "devops-poc01-prod"
aks_test_kube_config = <sensitive>
aks_test_name = "devops-poc01-test"
rg_name = "rg-devops-poc01"
```

2.6. Instalacja ArgoCD

Po uruchomieniu skryptu `install_argocd.sh`:

```
(You should delete the initial secret afterwards as suggested by the Getting Started Guide: https://argo-cd.readthedocs.io/en/stable/getting_started/#4-login-using-the-cli)
→ Czekam aż deployment 'argocd-server' będzie gotowy...
deployment "argocd-server" successfully rolled out
  ✔ argocd-server gotowy

→ Czekam na IP z LoadBalancera...
  ✔ Znalazłem IP po 1 próbach: 20.238.142.190
  🌐 ArgoCD URL (HTTP): http://20.238.142.190

→ Wyciągam hasło admina z sekretu 'argocd-initial-admin-secret'...
  👤 Login: admin
  🔑 Password: k11RUZkLmf0HyXy

→ Podsumowanie namespace'ów w klastrze devops-poc01-prod:
```

```
deployment "argocd-server" successfully rolled out
  ✔ argocd-server gotowy

→ Czekam na IP z LoadBalancera...
  ✔ Znalazłem IP po 1 próbach: 57.153.65.114
  🌐 ArgoCD URL (HTTP): http://57.153.65.114

→ Wyciągam hasło admina z sekretu 'argocd-initial-admin-secret'...
  👤 Login: admin
  🔑 Password: hJteS4Cln7rBwStt

→ Podsumowanie namespace'ów w klastrze devops-poc01-test:
argocd      Active   47s
```

Podsumowanie: Do tego momentu została postawiona kompletna infrastruktura wraz z gotowym ArgoCD wystawionym na świat z loginem i hasłem.

3. Platform-apps – GitOps i App-of-Apps

Repozytorium **platform-apps** realizuje wzorec **GitOps** z wykorzystaniem strategii **App-of-Apps**. W tym podejściu ArgoCD zarządza jedną główną aplikacją, która kontroluje wszystkie pozostałe aplikacje w klastrze.

3.1. Struktura projektu

```
platform-apps/
├── bootstrap/
│   ├── app-of-apps-prod.yaml
│   ├── app-of-apps-test.yaml
│   └── ingress-nginx.yaml
├── charts/
│   ├── app-of-apps/
│   │   ├── templates/
│   │   │   └── applications.yaml
│   │   └── values.yaml
│   └── deploy-platform.sh
└── get-access-info.sh
```

3.2. Folder bootstrap

Folder **bootstrap/** zawiera manifesty Kubernetes (Custom Resources), które instruuja ArgoCD, co ma zainstalować na początku:

- **app-of-apps-prod.yaml / app-of-apps-test.yaml** – Definicje zasobu typu **Application** wskazujące na folder **charts/app-of-apps**. Podział na prod/test umożliwia różne wersje aplikacji na różnych środowiskach.
- **ingress-nginx.yaml** – Manifest instalujący Ingress Controller, który pozwala na wystawienie DependencyTrack na zewnątrz (rozwiązuje problem błędu 405).

3.3. Folder charts/app-of-apps

To lokalny Helm Chart pełniący rolę "spisu treści" infrastruktury:

- **templates/applications.yaml** – Najważniejszy plik repozytorium. Zawiera pętlę, która na podstawie listy w **values.yaml** generuje kolejne obiekty **Application** dla ArgoCD (Prometheus, aplikacja backendowa, baza danych, itp.).
- **values.yaml** – Lista wszystkich aplikacji, które ArgoCD ma śledzić, wraz z adresami repozytoriów Git.

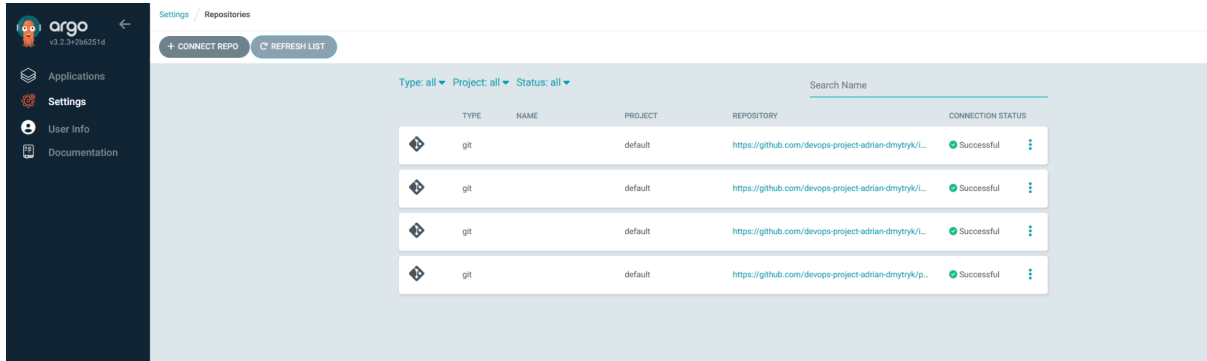
3.4. Skrypty automatyzacji

- **deploy-platform.sh** – Odpowiada za wdrożenie całego bootstrapu dla środowisk test i prod z poprawkami błędów Ingress

- **get-access-info.sh** – Wyświetla dane dostępne (hasło admina ArgoCD, adresy URL aplikacji)

Szczegółowy proces i opis działania znajduje się w:

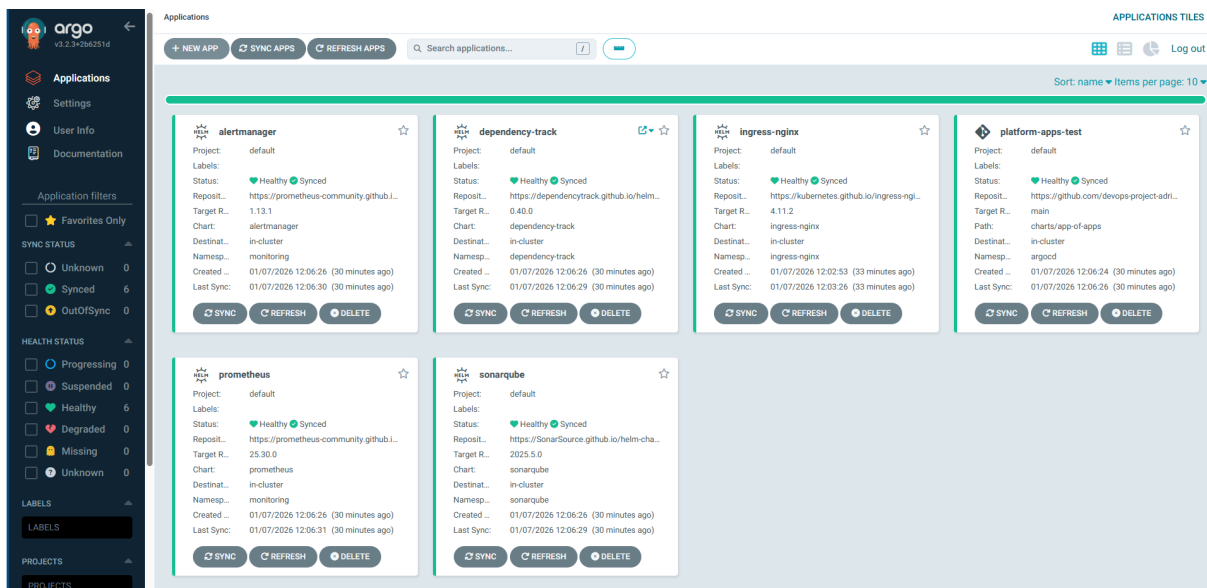
[Platform-apps/manuals/Deploy-platform.md](#)



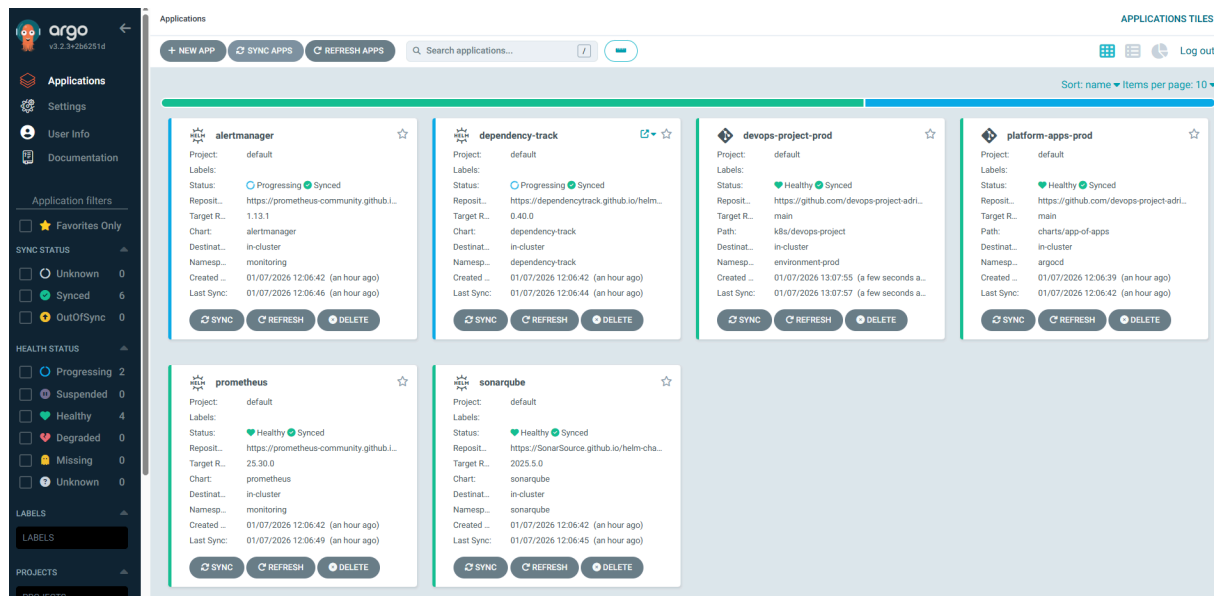
3.5. Rezultat wdrożenia

Po odpaleniu skryptu, tworzący się stack w ArgoCD:

Środowisko TEST:



Środowisko PROD:



4. Przygotowanie narzędzi CI/CD

4.1. Konfiguracja SonarQube

Proces konfiguracji SonarQube:

1. Pobranie adresu IP aplikacji i logowanie do UI danymi: **admin / admin**
2. Zmiana domyślnego hasła zgodnie z wymaganiami
3. Generowanie klucza API:
 - Ścieżka: **Administration** → **Access Management** → **Teams** → **Administrators** → **Generate New API Key**
4. Skopiowanie klucza API do późniejszego wykorzystania w CI

4.2. Konfiguracja DependencyTrack

Proces konfiguracji DependencyTrack:

1. Pobranie adresu IP aplikacji i logowanie do UI danymi: **admin / admin**
2. Zmiana domyślnego hasła zgodnie z wymaganiami
3. Generowanie klucza API:
 - Ścieżka: **Profil** → **My Account** → **Security** → wprowadzenie nazwy tokena → **Generate**
4. Skopiowanie klucza API do późniejszego wykorzystania

4.3. Konfiguracja kube-prometheus-stack

ArgoCD automatycznie wdraża cały stack wraz z regułami i sekretami wykorzystywanymi do wykrywania anomalii w aplikacji Java (np. błąd 500), które będą wysyłane na email.

Weryfikacja działania przez port-forward:

```
# Prometheus  
kubectl -n monitoring port-forward svc/prometheus-server 9090:80  
# → localhost:9090
```

```
# Alertmanager  
kubectl -n monitoring port-forward svc/alertmanager 9093:9093  
# → localhost:9093
```

Szczegółowy opis znajduje się w: [Platform-apps/manuals/Monitoring500Error.md](#)

5. Devops-project – Aplikacja i CI Pipeline

Repozytorium **devops-project** zawiera kod aplikacji Java Spring Boot wraz z pełnym pipeline CI automatyzującym procesy dostarczania oprogramowania.

5.1. Zakres CI Pipeline

Pipeline odpowiada za:

- Budowę i testowanie aplikacji Java
- Analizę jakości kodu z wykorzystaniem SonarQube
- Skanowanie bezpieczeństwa za pomocą Trivy
- Generowanie SBOM w formacie CycloneDX
- Budowę i publikację obrazu Docker do Azure Container Registry (ACR)
- Automatyczną promocję wersji do środowiska DEV (GitOps z ArgoCD)

5.2. Etapy wykonania

Pipeline w GitHub Actions uruchamia się automatycznie po każdym pushu na gałąź **main**.

1. Build & Test

- Uruchomienie **mvn clean verify**
- Artefakty testowe dostępne w GitHub Actions → Artifacts

2. Analiza jakości SonarQube

- Skan i analiza jakości kodu
- Raport dostępny w instancji SonarQube

3. Bezpieczeństwo i SBOM

- Generowanie SBOM (**bom.json**) w formacie CycloneDX

- Skan bezpieczeństwa obrazu Docker za pomocą Trivy
- Wysłanie SBOM do Dependency-Track przez REST API
- Rejestracja wersji aplikacji w Dependency-Track

4. Docker Build & Push

- Budowanie obrazu Docker z tagiem `GITHUB_SHA`
- Publikacja do Azure Container Registry (ACR)

5. Promocja do środowiska DEV

- Aktualizacja `values/devops-project/values.yaml` w repozytorium `infrastructure-env-dev`
- Automatyczne utworzenie Pull Request (PR)
- Po zatwierdzeniu PR → ArgoCD wykonuje rollout na DEV

5.3. Przepływ CI/CD

Commit do devops-project/main



GitHub Actions CI Pipeline



Build & Test



SonarQube (Analiza jakości)



Security & SBOM (Trivy + CycloneDX + Dependency-Track)



Docker Build & Push (ACR)



Promocja do DEV (infrastructure-env-dev + GitOps ArgoCD)

5.4. Konfiguracja sekretów GitHub

Repository secrets

New repository secret

Name ↕	Last updated		
 ACR_LOGIN_SERVER	2 months ago		
 ACR_NAME	last month		
 AZURE_CREDENTIALS	2 months ago		
 DEPENDENCY_TRACK_API_KEY	26 minutes ago		
 DTRACK_API_URL	7 minutes ago		
 ENV_REPOS_TOKEN	2 months ago		
 GH_PAT_GITOPS	2 months ago		
 SONAR_HOST_URL	11 minutes ago		
 SONAR_TOKEN	14 minutes ago		

Azure (Chmura)

- **AZURE_CREDENTIALS** – Generowane komendą `az ad sp create-for-rbac`. Dane Service Principal umożliwiające GitHubowi tworzenie infrastruktury w Azure
- **ACR_NAME / ACR_LOGIN_SERVER** – Nazwa rejestru kontenerów (np. `acrfordevops poc01adrian.azurecr.io`)

SonarQube (Analiza kodu)

- **SONAR_TOKEN** – Generowany w: `My Account` → `Security` → `Generate Token`
- **SONAR_HOST_URL** – Adres URL instancji SonarQube na klastrze AKS

Dependency Track (Bezpieczeństwo bibliotek)

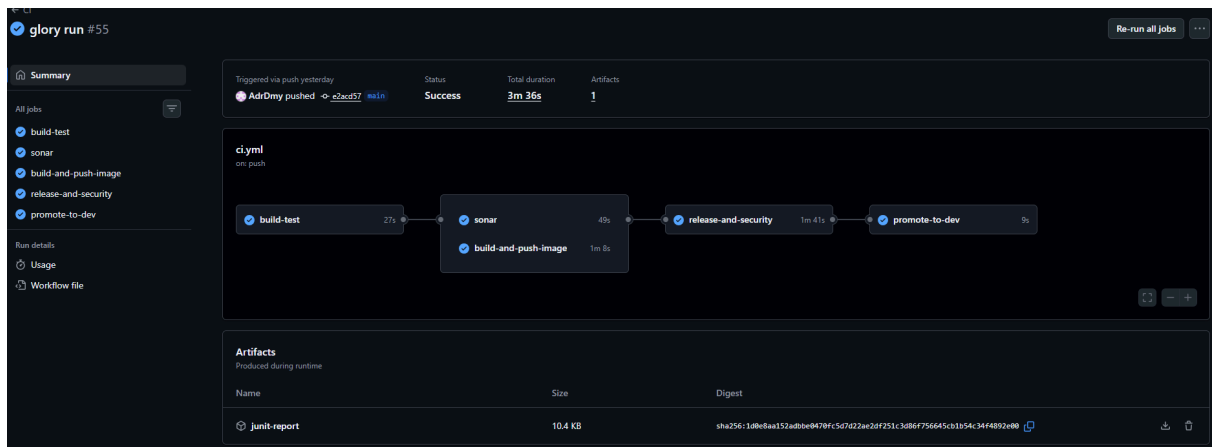
- **DEPENDENCY_TRACK_API_KEY** – Generowany w: `Administration` → `Teams` → `Automation` → `API Key`
- **DTRACK_API_URL** – Adres API instancji Dependency Track

GitHub (Integracja GitOps)

- **GH_PAT_GITOPS / ENV_REPOS_TOKEN** – Personal Access Token (PAT) generowany w: `Settings` → `Developer Settings` → `Personal Access Tokens`. Umożliwia pipeline'owi CI pushowanie zmian do repozytoriów środowiskowych.

5.5. Uruchomienie CI

Po wykonaniu commit i push do Gita, GitHub Actions wykrywa zmiany i rozpoczyna proces CI:

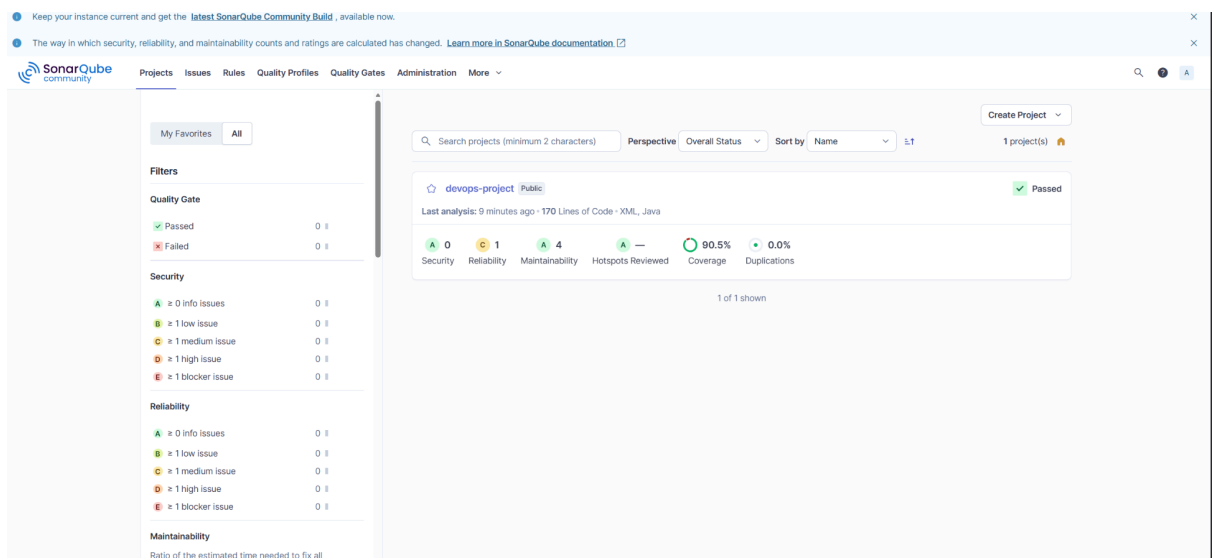


Agent GitHub Actions przechodzi przez cały proces i tworzy commit na repozytorium **infrastructure-env-dev** z nowym obrazem.

5.6. Wyniki analizy

SonarQube – Pokrycie kodu

a)



DependencyTrack – Skanowanie zagrożeń

Dashboard

PORTFOLIO

Projects

Components

Vulnerabilities

Licenses

Tags

GLOBAL AUDIT

Vulnerability Audit

Policy Violation Audit

ADMINISTRATION

Policy Management

Administration

Home / Projects

0

Portfolio Vulnerabilities

0

Projects at Risk

0

Vulnerable Components

0

Inherited Risk Score

+ Create Project

☐ Show inactive projects

☐ Show flat project view

Search

↺

≡

Project Name	Version	Latest	Classifier	Last BOM Import	BOM Format	Risk Score	Active	Policy Violations	Vulnerabilities
devops-project	test-local		Library	30 Dec 2025 at 14:20:04	CycloneDX 1.6	0	☒	0	0
devops-project	e5a0aeb4b65c0ca11281a9119281c983c1db2beb		Library	30 Dec 2025 at 15:10:24	CycloneDX 1.6	0	☒	0	0
test-project	1.0.0		Library	30 Dec 2025 at 15:04:58	CycloneDX 1.6	0	☒	0	0

Showing 1 to 3 of 3 rows

Home / Projects / devops-project • e5a0aeb4b65c0ca11281a9119281c983c1db2beb

devops-project

e5a0aeb4b65c0ca11281a9119281c983c1db2beb

0

0

0

0

0

View Details >

Overview

Components

Services

Dependency Graph

Audit Vulnerabilities

Exploit Predictions

Policy Violations

+ Add Component

- Remove Component

Upload BOM

Download BOM

Download Components

☐ Outdated only

☐ Direct only

Search

↺

≡

Component	Version	Group	Internal	License	Risk Score	Vulnerabilities
HdrHistogram	2.2.2	org.hdrhistogram		CC0-1.0	0	0
LatencyUtils	2.0.3	org.latencyutils		CC0-1.0	0	0
jackson-annotations	2.18.2	com.fasterxml.jackson.core		Apache-2.0	0	0
jackson-core	2.18.2	com.fasterxml.jackson.core		Apache-2.0	0	0
jackson-databind	2.18.2	com.fasterxml.jackson.core		Apache-2.0	0	0
jackson-datatype-jdk8	2.18.2	com.fasterxml.jackson.datatype		Apache-2.0	0	0
jackson-datatype-jsr310	2.18.2	com.fasterxml.jackson.datatype		Apache-2.0	0	0
jackson-module-parameter-names	2.18.2	com.fasterxml.jackson.module		Apache-2.0	0	0
jakarta.annotation-api	2.1.1	jakarta.annotation		EPL-2.0	0	0
jul-to-slf4j	2.0.16	org.slf4j		MIT	0	0

Showing 1 to 10 of 48 rows

10

rows per page

1

2

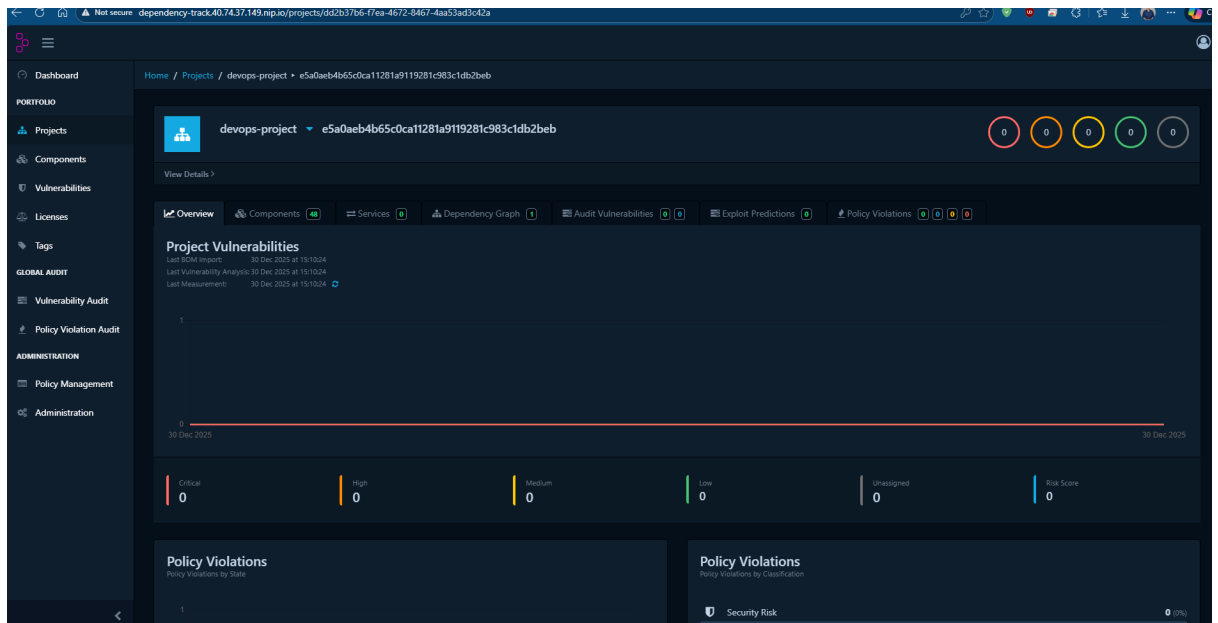
3

4

5

49 nip.io/#

Dependency-Track v4.13.6



6. Infrastructure-env-dev – Środowisko deweloperskie

6.1. Zawartość repozytorium

Repozytorium zawiera konfigurację środowiska **DEV**:

- `values/devops-project/values.yaml` – wartości Helm/Customize dla DEV
- `k8s/devops-project/` – manifesty aplikacji (Deployment, Service)
- `deploy-argo-dev.sh` – uruchamia aplikację w ArgoCD na środowisku test
- Workflow promotion: **DEV** → **TEST**

6.2. Proces GitOps

Po merge PR w tym repozytorium:

1. ArgoCD w środowisku DEV wykrywa nowy commit
2. Wykonuje auto-sync
3. Rolloutuje nową wersję aplikacji do namespace `environment-dev`

6.3. Promocja do TEST

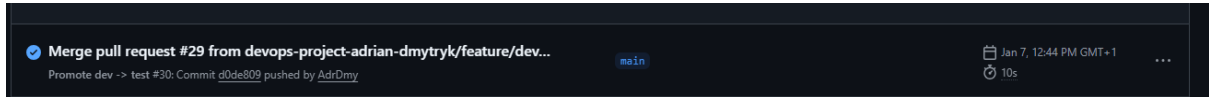
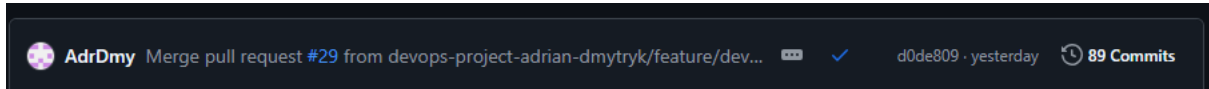
Workflow `promote-to-test`:

1. Pobiera aktualny tag z `values/devops-project/values.yaml`
2. Aktualizuje ten sam tag w repozytorium `infrastructure-env-test`
3. Tworzy Pull Request do TEST
4. Merge PR → ArgoCD TEST wykonuje rollout

Powiązane repozytoria:

- `devops-project` (źródło wersji)
- `infrastructure-env-test` (cel promocji)

Stan po merge:



7. Infrastructure-env-test – Środowisko testowe

7.1. Zawartość repozytorium

Repozytorium zawiera deklaratywną konfigurację środowiska **TEST**:

- `values/devops-project/values.yaml` – wartości aplikacji dla TEST
- `k8s/devops-project/` – manifesty Kubernetes dla TEST
- Workflow promotion: **TEST** → **PROD**

7.2. Proces GitOps

Po merge PR:

1. ArgoCD synchronizuje repozytorium
2. Deployuje nową wersję do namespace `environment-test`

7.3. Promocja do PROD

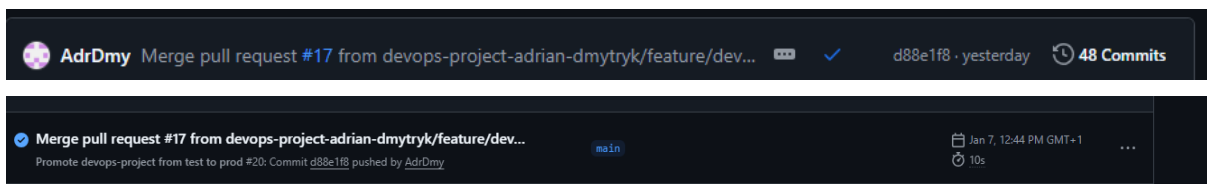
Workflow `promote-to-prod`:

1. Odczytuje tag z `values/devops-project/values.yaml`
2. Wprowadza ten sam tag do repozytorium `infrastructure-env-prod`
3. Tworzy PR do PROD
4. Merge PR → rollout na production

Powiązane repozytoria:

- `infrastructure-env-dev`
- `infrastructure-env-prod`

Stan po wykonaniu CD:



8. Infrastructure-env-prod – Środowisko produkcyjne

8.1. Zawartość repozytorium

Repozytorium zawiera konfigurację środowiska produkcyjnego:

- `values/devops-project/values.yaml`
- `k8s/devops-project/`
- Walidację manifestów (syntactic check)
- Finalny etap łańcucha promocji

8.2. Proces GitOps

Po merge PR:

1. ArgoCD w PROD wykonuje auto-sync
2. Następuje rollout aplikacji w namespace `environment-prod`

8.3. Walidacja manifestów

Workflow wykonuje walidację składniową:

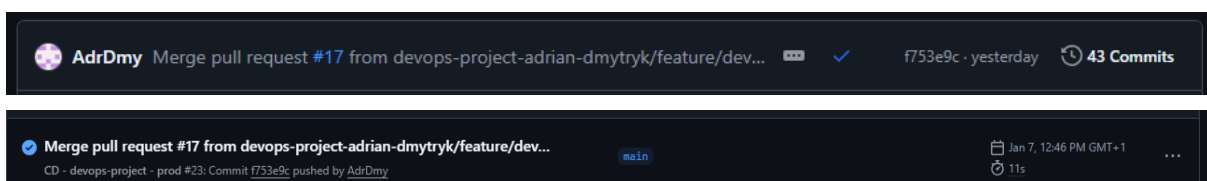
```
kubectl apply --dry-run=client --validate=false
```

Ta walidacja nie wymaga podłączania się do klastra.

Powiązane repozytoria:

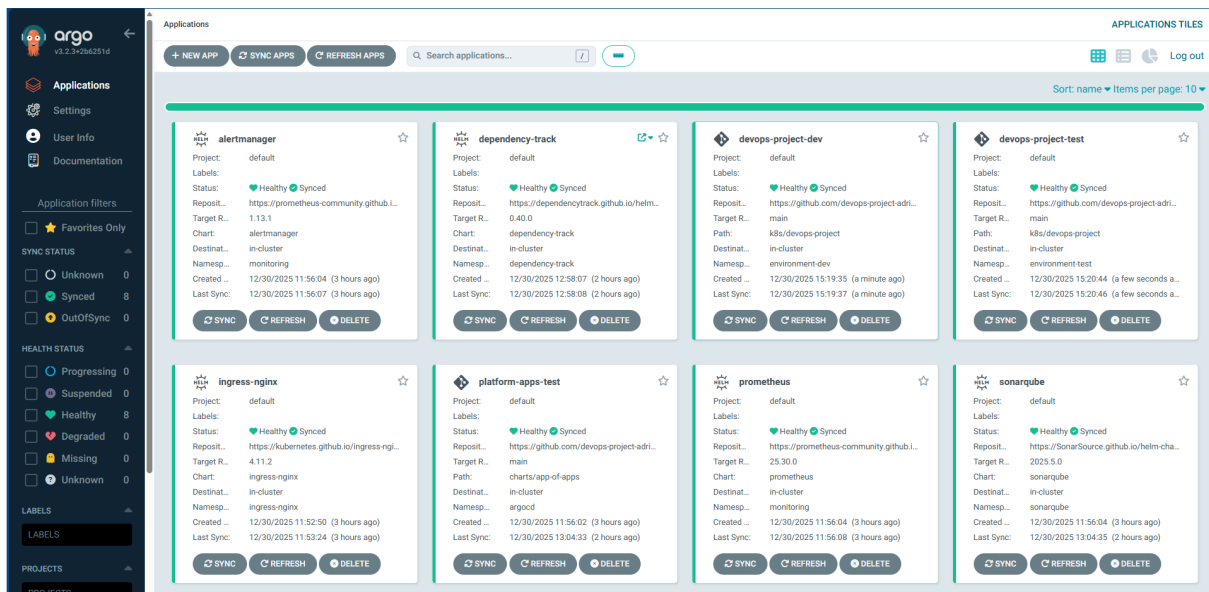
- `infrastructure-env-test`

Stan po CD:

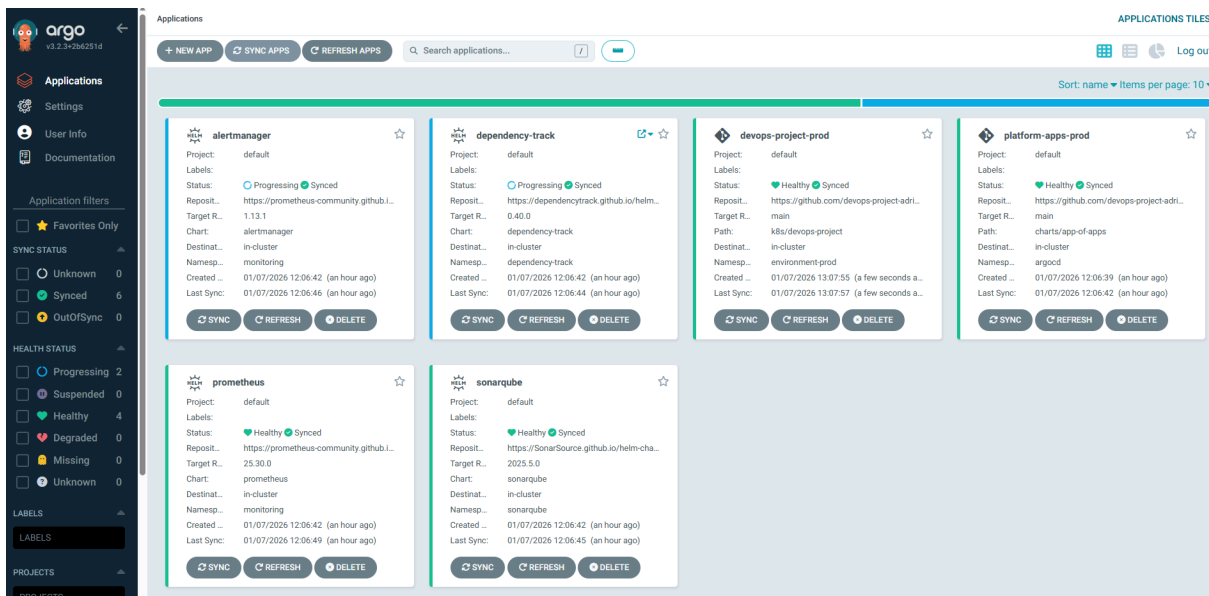


Wdrożone aplikacje po promocji

Środowisko TEST:



Środowisko PROD:



9. Monitoring aplikacji

System monitoringu wykrywa błędy 500 w aplikacji Java i wysłał powiadomienia email.

9.1. Kluczowe komponenty

- **Aplikacja** – Spring Boot z włączonym Actuator, wystawiającym metryki Prometheus przez `/actuator/prometheus`
- **Monitoring** – Prometheus (zbieranie metryk) + Alertmanager (zarządzanie powiadomieniami)
- **Orkiestracja** – ArgoCD (GitOps) + Helm (zarządzanie konfiguracją i wersjami)

9.2. Instrukcja konfiguracji

Po wdrożeniu nowej infrastruktury, wykonaj następujące kroki:

1. Synchronizacja i czyszczenie pamięci konfiguracyjnej

- Utwórz Secret `alertmanager-gmail` w namespace `monitoring`
- W ArgoCD wykonaj synchronizację z opcją `Replace`

Zrestartuj Prometheus:

```
kubectl rollout restart deployment prometheus-server -n monitoring
```

•

2. Wystawienie usług (Port-Forwarding)

Uruchom w osobnych terminalach:

Aplikacja

```
kubectl -n environment-dev port-forward svc/devops-project 8080:80
```

```
# → localhost:8080
```

Prometheus

```
kubectl -n monitoring port-forward svc/prometheus-server 9090:80
```

```
# → localhost:9090
```

Alertmanager

```
kubectl -n monitoring port-forward svc/alertmanager 9093:9093
```

```
# → localhost:9093
```

3. Generowanie błędów (Test obciążeniowy)

Uruchom pętlę testową:

```
while true; do
  curl -s http://localhost:8080/api/error500 > /dev/null
  echo "Wysłano żądanie błędu: $(date)"
  sleep 2
done
```

9.3. Checklist weryfikacji

✓ Metryki aplikacji

- URL: `http://localhost:8080/actuator/prometheus`
- Szukaj: `http_server_requests_seconds_count{status="500", ...}` – licznik musi rosnać

✓ Połączenie Prometheus → Alertmanager

- URL: `http://localhost:9090/status`
- Sekcja Alertmanagers: `http://alertmanager:9093/...`
- ⚠ Jeśli widzisz IP (np. `10.0.x.x`) – zrestartuj Prometheus!

✓ Stan alertu

- URL: `http://localhost:9090/alerts`
- Alert `DevopsProjectHttp500` = **Firing** (czerwony)

✓ Logi wysyłki email

`kubectl logs alertmanager-0 -n monitoring --tail=20`

Szukaj: `component=dispatcher msg="Notify success" receiver=gmail`

✓ Gmail

Sprawdź skrzynkę: `adrian.dmytryk@gmail.com` (Odebrane i SPAM)

9.4. Rezultaty monitoringu

Screeny po wywołaniu błędu 500:

The top screenshot shows the Prometheus 'Status' tab. It displays a table of scrape targets for the 'serviceMonitor/monitoring/devops-project/0' endpoint. The table includes columns for 'Endpoint', 'Labels', 'Last scrape', and 'State'. The 'State' column shows 'UP' with a green dot.

The bottom screenshot shows the Prometheus 'Alerts' tab. It displays a list of active alerts. The first alert is 'DevopsProjectHttp500' (1 active), which is in a 'FIRING' state. The alert details show the name, expression, labels, severity, and annotations. The 'State' column shows 'FIRING' in red.

Labels	State	Active Since	Value
alertname=DevopsProjectHttp500 application=devops-project error=none exception=none instance=devops-project-685f6cd785-744ts job=devops-project method=GET namespace=SERVER-13830 severity=critical status=500 url=http://localhost:8080	FIRING	2026-01-07T12:30:22.013712351Z	57.02946666666667

devops-project.rules

/etc/prometheus/rules/prometheus-monitoring-kube-prometheus-prometheus-rulefiles-0/monitoring-devops-project-http500-78254d5-06ab-8c49-80af5bb097c9.yaml

FIRING (1)

DevopsProjectHttp500

FIRING (1)

For: 1m

severity: "critical"

description

Wykryto co najmniej jeden HTTP 500 w ciągu 5 minut

summary

devops-project zwraca HTTP 500

Alert labels

labelname	labelvalue	State	Active Since	Value
alertname	"DevopsProjectHttp500"	FIRING	1m 8.739s	42.66096
severity	"critical"			

description

Wykryto co najmniej jeden HTTP 500 w ciągu 5 minut

summary

devops-project zwraca HTTP 500

Rules		Interval: 1m 0s		15.342s ago	0.814ms
devops-project					
Rule		State	Error	Last Evaluation	Evaluation Time
alert: DevopsProjectHttp500 expr: increase(http_server_requests_seconds_count(status="500"))[5m] > 0 labels: severity: critical annotations: description: Wykryto błąd 500 summary: HTTP 5xx detected in devops-project		OK		15.346s ago	0.787ms

[FIRING:1] (DevopsProjectHttp500 devops-project none none devops-project.environment-dev.svc.cluster.local:80 spring-boot GET SERVER_ERROR critical 500 /api/error500)

Utwórz podsumowanie tego e-maila

adriandmytryk@gmail.com 15:41 (1 minutę temu)

1 alert for

View In Alertmanager

[1] Firing

Labels

alertname = DevopsProjectHttp500
application = devops-project
error = none
exception = none
instance = devops-project.environment-dev.svc.cluster.local:80
job = spring-boot
method = GET
outcome = SERVER_ERROR
severity = critical
status = 500
uri = /api/error500

Annotations

description = Wykryto błąd 500
summary = HTTP 5xx detected in devops-project



10. Dokumentacja projektu

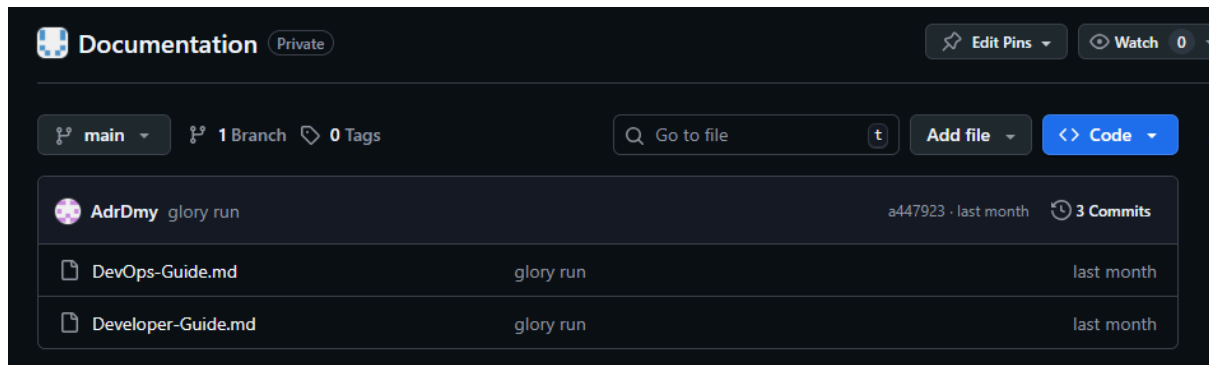
Cała dokumentacja znajduje się w repozytorium **Documentation** i dzieli się na:

a) Dokumentacja dla DevOpsa / Architekta

[DevOps-Guide.md](#)

b) Dokumentacja dla Developera

[Developer-Guide.md](#)



Powiązane repozytoria

- **infra-azure** – Infrastruktura jako kod (Terraform)
- **platform-apps** – GitOps, App-of-Apps, narzędzia platformowe
- **devops-project** – Aplikacja Java Spring Boot + CI Pipeline
- **infrastructure-env-dev** – Konfiguracja ArgoCD dla środowiska DEV
- **infrastructure-env-test** – Konfiguracja ArgoCD dla środowiska TEST
- **infrastructure-env-prod** – Konfiguracja ArgoCD dla środowiska PROD
- **Documentation** – Dokumentacja projektu

Projekt zrealizowany przez Adriana Dmytryka